

[← Back to articles](#)

ARTICLES

Inside Claude Code: The Leaked Source Analysis

Anthropic's leaked 512k-line Claude Code source reveals how production AI agents work with thin loops, deep security, and hidden features.

o-mega [Get started](#)



O-mega Team

31 March 2026 • 50 min read

A deep technical analysis of what Anthropic's leaked 512,000 lines of TypeScript reveal about how production AI agent harnesses actually work.

Written by Yuma Heymans (@yumahey), who has been writing code since age six and now builds AI agent infrastructure at o-mega.ai. He reverse-engineers agent harnesses like this one to inform how the next generation of autonomous AI systems should be built.

On March 31, 2026, security researcher Chaofan Shou discovered a 57MB source map file buried inside the npm package `@anthropic-ai/claude-code version 1.0.33`. That single `.map` file contained URLs pointing to the complete, unobfuscated TypeScript source code hosted on Anthropic's own R2 cloud storage bucket. Within hours, his X post had garnered **3.1 million**

views, the code was mirrored to multiple GitHub repositories, and the developer community had the most detailed look ever at how a production AI coding agent actually works under the hood - [CyberSecurityNews](#).

The numbers are staggering. **1,900 TypeScript files. 512,000+ lines of code. 40+ built-in tools, 50+ slash commands, and 32 compile-time feature flags** hiding unreleased features including an always-on assistant, a Tamagotchi-style companion pet, and a remote planning engine that can run for 30 minutes straight. All exposed because someone forgot to add `*.map` to `.npmignore`.

What makes this leak genuinely valuable (and not just another security incident to tut about on Hacker News) is what it reveals about the fundamental engineering problem of our moment: how do you take a powerful language model and wrap it in enough scaffolding to interact with the real world safely, efficiently, and at scale? Claude Code is one of the most polished, commercially deployed agent harnesses in production today, serving hundreds of thousands of developers. Its source code is a blueprint for how that wrapping actually works.

This guide goes deep into the actual code. Not summaries of summaries. The actual `query.ts`, `main.tsx`, `bashSecurity.ts`, and `companion.ts` files. We will trace the execution path from user prompt to tool call and back, examine the permission system that prevents the model from destroying your machine, and reveal what the hidden feature flags tell us about where AI coding assistants are headed.

Contents

1. The Leak: How a Build Config Mistake Exposed Everything
2. The Numbers: What 512,000 Lines Actually Looks Like

3. The Core Loop: 50 Lines That Run the Whole Thing
4. Four Primitives Instead of 100 Integrations
5. Context Management: Five Compaction Strategies
6. The Six-Layer Memory System
7. Security: 2,592 Lines of Bash Paranoia
8. The YOLO Classifier and Auto-Mode Permissions
9. Sub-Agents: Isolated Forked Intelligence
10. Hidden Features: KAIROS, ULTRAPLAN, and the Buddy System
11. Code Quality: The "Vibe Coded" Debate
12. What This Means for Agent Harness Design
13. Where This All Goes Next

1. The Leak: How a Build Config Mistake Exposed Everything

The root cause was comically simple. Claude Code uses **Bun** as its JavaScript bundler. Bun generates source maps by default in production builds. Source maps are `.map` files that map minified/bundled code back to the original source, and they are essential during development for debugging but should never ship in production packages. Anthropic's build pipeline failed to exclude these files from the npm distribution - [DEV Community](#).

The `.map` files did not contain the source code directly. They contained URLs pointing to Anthropic's R2 storage bucket where the original TypeScript files lived, and those URLs were publicly accessible. Chaofan Shou followed the URLs, reconstructed the codebase, and disclosed the finding. Multiple developers created public GitHub mirrors within hours. The most prominent, [Kuberwastaken/claude-code](#), accumulated over **1,100 stars** and **1,900 forks** before Anthropic could respond - [Bytelota](#).

The irony was not lost on the community. An AI coding assistant, a tool whose entire value proposition is helping developers write better code and avoid mistakes, had shipped with a basic build hygiene oversight. The [Hacker News thread](#) that followed was equal parts architectural analysis and comedy.

But here is the real story. According to the [Pragmatic Engineer](#), Claude Code's origin was a prototype by Boris Cherny in September 2024 that started by using AppleScript to report what music was playing. Adding filesystem access was the breakthrough. Internal dogfooding started November 2024, with **20% of Anthropic Engineering** adopting it on day one and **50% within five days**. By the time of the leak, Anthropic claimed that approximately **90% of Claude Code's codebase was written by Claude Code itself**. The tool had been releasing at a pace of **60-100 internal builds daily**, with one external npm release per day. Engineers averaged around **5 PRs daily** during peak development, with a **67% increase** in PR throughput even as the team doubled in size.

This context matters. What leaked was not a carefully architected system designed by a committee of senior engineers. It was a rapidly iterated product, largely self-written by an AI, that was shipping at startup velocity. That explains both its architectural elegance in some areas and its messiness in others.

The community response was divided. On one hand, developers praised the architectural decisions: the thin TAOR loop, the composable tool system, the defense-in-depth permission model. On the other, many were critical of code quality, pointing to massive files and deep nesting as evidence of "vibe coding." Some developers used Claude Code itself to reverse-engineer its own source, creating a recursive irony that the Hacker News crowd thoroughly enjoyed. Multiple independent analyses were published within days, including a detailed architecture breakdown by [Vrungta on Substack](#),

a reverse-engineering walkthrough by [Sabrina Ramonov](#), and a technical internals deep dive by [Kir Shatrov](#).

Notably, the leak also exposed **Anthropic's internal project codename** for Claude Code: **"Tengu"**. This name appears hundreds of times as a prefix for feature flags and analytics events (e.g., `tengu_startup_telemetry`, `tengu_ultraplan_model`, `tengu_onyx_plover`). The other animal codenames discovered (Capybara, Fennec) likely refer to unreleased model variants. These internal naming conventions, combined with the Undercover Mode that prevents their public disclosure, paint a picture of a company that takes operational security seriously, which makes the source map leak all the more ironic.

For readers who want to explore the code themselves, the most actively maintained mirror is the [Kuberwastaken/claude-code](#) repository, which includes a structured breakdown of the major components. Additional analysis tools have emerged, including [Yuyz0112/claude-code-reverse](#) for visualizing LLM interactions, and a dedicated catalog site at [CCLeaks.com](#) documenting hidden features. The system prompts have been extracted and published separately at the [Piebald-AI repository](#).

2. The Numbers: What 512,000 Lines Actually Looks Like

Before diving into architecture, the structural overview gives you a sense of how the codebase is organized. This comes from our own analysis of the local source:

The `src/` directory contains all **1,884 TypeScript files** across **55+ directories**. Here is where the code actually lives:

Directory	Files	Lines	Purpose
utils/	564	180,472	Core utilities: file ops, settings, permissions, tokens, git
components/	389	81,546	React/Ink terminal UI components
services/	130	53,680	External integrations: MCP, API, OAuth analytics
tools/	184	50,828	Tool implementations (40+ tools)
commands/	189	26,428	Slash command implementations (~5 commands)
hooks/	104	19,204	Permission, state, and lifecycle hooks
cli/	19	12,353	CLI entry points including <code>print.ts</code> (5,594 lines)
bridge/	31	12,613	IDE bridge (VS Code, JetBrains integration)
buddy/	6	~500	The companion pet system

Two things jump out immediately. First, **utils/ contains 35% of the entire codebase**. This is where the real engineering lives: permissions, security validation, file state caching, token counting, git integration, settings management, and the startup profiler. Second, the **tools/ directory** at 50,828 lines is larger than most entire applications. Each tool (Bash, Read,

Write, Edit, Grep, Glob, Agent, MCP, Web, etc.) is a self-contained module with its own schema, permission logic, and execution flow.

The technology stack is telling. **TypeScript** on **Bun** runtime. **React with Ink** for the terminal UI (using Meta's Yoga flexbox engine for layout). **Zod v4** for schema validation. The Anthropic SDK for API calls. The MCP SDK for protocol communication. And **GrowthBook** for feature flag management and analytics. Heavy dependencies like OpenTelemetry and gRPC are lazy-loaded to keep startup fast, with parallel prefetching of macOS MDM settings and keychain reads firing before module evaluation even begins.

The very first lines of `main.tsx` show this startup optimization in action:

```
// These side-effects must run before all other imports:
// 1. profileCheckpoint marks entry before heavy module evaluation begins
// 2. startMdmRawRead fires MDM subprocesses (plutil/reg query) so they run
//    parallel with the remaining ~135ms of imports below
// 3. startKeychainPrefetch fires both macOS keychain reads (OAuth + legacy
//    key) in parallel
profileCheckpoint('main_tsx_entry');
startMdmRawRead();
startKeychainPrefetch();
```



This is production engineering. Every millisecond of startup time matters for a CLI tool that developers invoke hundreds of times a day. The code fires off I/O operations as side effects before import statements even finish evaluating, hiding network and disk latency behind JavaScript module loading time.

3. The Core Loop: 50 Lines That Run the Whole Thing

The most important architectural decision in Claude Code lives in `src/query.ts`, starting at line 241. The core runtime is an async generator function called `queryLoop` that implements what the codebase calls the **TAOR pattern**: Think, Act, Observe, Repeat. This is the engine that powers every interaction, from a simple "read this file" to a complex multi-step refactoring across 50 files.

Here is the actual structure from the source:

```
async function* queryLoop(  
  params: QueryParams,  
  consumedCommandUuids: string [],  
) : AsyncGenerator<StreamEvent | Message | TombstoneMessage> {  
  let state: State = {  
    messages: params.messages,  
    toolUseContext: params.toolUseContext,  
    maxOutputTokensOverride: params.maxOutputTokensOverride,  
    autoCompactTracking: undefined,  
    stopHookActive: undefined,  
    maxOutputTokensRecoveryCount: 0,  
    hasAttemptedReactiveCompact: false,  
    turnCount: 1,  
    pendingToolUseSummary: undefined,  
    transition: undefined,  
  }  
  
  while (true) {  
    // THINK: Assemble context, system prompt, tools  
    // ACT: Stream API call to Claude model
```

```
// OBSERVE: Execute tool calls, collect results
// REPEAT: Append results to messages, loop back
}
}
```

The `while (true)` loop is deliberate. It runs indefinitely, terminated only by explicit return statements when the model decides it is done (no more tool calls), when an error occurs, or when a budget is exhausted. The mutable `state` object carries everything across iterations: the conversation messages, tool execution context, compaction tracking, and turn counting.

The function is an **async generator**, which is the key architectural choice. Instead of returning a final result, it `yield s` events as they happen: streaming text chunks, tool progress updates, compaction boundaries, and final messages. This means the UI (the React/Ink terminal renderer) gets real-time updates without blocking. When the model starts streaming a response, the user sees text appearing character by character. When a tool starts executing, a progress indicator shows immediately.

Within each iteration of the loop, the actual sequence is:

1. **Context assembly:** Slash commands are dequeued. Memory prefetch results are consumed. Skill discovery runs in the background. The system prompt is assembled from cached and dynamic components, split at a `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` marker that separates globally-cacheable content from per-session content (reducing API input tokens through prompt caching).
2. **API streaming call:** The assembled prompt, tools, and messages are sent to the Claude API via `deps.callModel()`. The response streams back as content blocks: text, thinking blocks, and `tool_use` blocks.

3. **Tool execution:** When the model emits `tool_use` blocks, a `StreamingToolExecutor` runs tools in parallel with the streaming response. This means the model can keep generating text while a file read or bash command executes in the background.
4. **Result assembly:** Tool results are packaged as `tool_result` blocks and appended to the message history. The loop checks stop conditions (`end_turn`, budget exhaustion, user interruption) and either returns or continues.

The comment in the source captures the philosophy: **"Mutable cross-iteration state. The loop body destructures this at the top of each iteration so reads stay bare-name. Continue sites write `state = { ... }` instead of 9 separate assignments."** This is production code optimized for developer velocity, not textbook cleanliness.

What makes this design philosophically important is what it does NOT do. There is no planning step. No workflow graph. No state machine. No explicit decision tree about which tools to use. The model receives context and decides what to do. The harness provides capabilities and constraints, but the intelligence lives entirely in the model. As the [Pragmatic Engineer analysis](#) revealed, Anthropic's internal codeword for this system is **"nO"**, and the design philosophy is captured by the phrase: **"Delete code when models improve."**

Every time a new model ships, the team deletes scaffolding. With the Claude 4.0 models, they reportedly **deleted around half the system prompt**. The harness is designed to shrink. This is the opposite of how most software evolves, and it reflects a bet that model capability improves faster than framework complexity can keep up.

We explored this fundamental tension between thin loops and thick orchestration in our [guide to making LLMs autonomous](#). Claude Code's

architecture is the clearest production validation of the thin-loop approach.

The streaming architecture deserves special attention because it solves a real UX problem. The `queryLoop` function is an **async generator** that yields `StreamEvent` objects as they happen. When the model starts producing text, the user sees characters appearing in real time. When a tool begins executing, a progress indicator shows immediately. When compaction triggers, the UI receives a `compact_boundary` event. This is not just cosmetic. Real-time feedback keeps the developer engaged and informed during multi-step operations that might take minutes. Without streaming, a complex refactoring task would show nothing for 30 seconds, then dump a wall of results.

There is also a fascinating component called the **h2A asynchronous queue** that enables mid-task user interjections. This dual-buffer queue achieves zero-latency delivery when a consumer is already waiting (the `enqueue()` call resolves the pending Promise immediately, bypassing internal buffering), but buffers messages when no reader is active. It implements `Symbol.asyncIterator` for `for await` consumption and enforces single-consumer semantics. The practical effect is that users can interrupt Claude Code mid-execution, steer it in a different direction, or cancel operations, and the system responds immediately rather than waiting for the current tool to complete.

The query engine also implements **multi-model fallback logic**. When an API call fails (rate limiting, server error, prompt too long), the system can fall back from a heavy model to a lighter one (for example, from Sonnet to Haiku) and retry. There is an exponential backoff retry wrapper, a configurable maximum retry count, and specific handling for prompt-too-long errors that trigger reactive compaction before retrying. The `maxOutputTokensRecoveryCount` in the state object tracks how many times the system has attempted to recover from output truncation, with a hard cap at 3 retries.

Token budgeting runs throughout the loop. Behind the `TOKEN_BUDGET` feature flag, a `budgetTracker` monitors cumulative input and output token usage, enforcing hard limits on cost. There is also a `taskBudgetRemaining` counter that tracks budget across compaction boundaries. The comment in the source is instructive: **"After a compact, the server sees only the summary and would under-count spend; remaining tells it the pre-compact final window that got summarized away."** This level of detail in cost management reflects the reality that Claude Code usage at scale represents significant API spend, and customers need predictable billing.

4. Four Primitives Instead of 100 Integrations

Rather than building dedicated integrations for GitHub, Jira, Slack, databases, and cloud providers, Claude Code provides **four fundamental capability primitives** that compose into arbitrary workflows:

Read (file reading, image viewing, PDF parsing), **Write/Edit** (file creation and precise string replacement), **Execute** (arbitrary shell commands via Bash), and **Connect** (extend capabilities through MCP servers).

The tool system itself is substantial. The `src/Tool.ts` file defines the core interface:

```
export type Tool = {
  name: string
  description: string
  input: ToolInputJSONSchema
  isAllowed: (permission: ToolPermissionContext) => boolean
  isVisible?: (options: { ... }) => boolean
  execute: (
```

```
    input: unknown,  
    context: ToolUseContext,  
  ) => AsyncGenerator<ToolProgressData | Message, ToolResult, unknown>  
}
```

Every tool is an async generator (matching the streaming architecture of the core loop), has a Zod-validated input schema, and includes permission checking as a first-class property. The `ToolUseContext` that gets passed to every tool execution is enormous, carrying the full conversation state, MCP client connections, abort controllers, file state cache, attribution tracking, and more.

One subtle but important detail: **tools are sorted alphabetically before being submitted to the API**. This is not for human readability. It is for **prompt cache hit rate optimization**. Since Claude's API caches prompt prefixes, keeping the tool definitions in a deterministic order across requests means more of the prompt gets served from cache, reducing both latency and cost.

The 18 core built-in tools break into clear categories. Search and discovery: **Bash**, **Glob** (fast file pattern matching), **Grep** (regex search via ripgrep), **LS**, and **Agent** (sub-agent dispatch). File operations: **Read**, **Edit**, **MultiEdit**, **Write**, and **NotebookEdit**. Web and information: **WebFetch**, **WebSearch**, **TodoRead/TodoWrite**. And the extensibility layer: **BatchTool** (parallel/serial multi-tool execution) and **ToolSearchTool** (deferred tool discovery for large tool sets).

MCP (Model Context Protocol) is the extension mechanism. The `src/services/mcp/` directory handles server discovery, connection management across multiple transport types (stdio, SSE, HTTP, SDK Control), and dynamic tool registration. MCP servers can be configured from eight different scopes: user config files, CLI settings, environment

variables, organization policies, project `.claude` files, and Claude.ai account settings. When a developer connects a custom MCP server, its tools appear alongside the built-in ones seamlessly.

This composability approach trades coverage for power. A dedicated GitHub integration might support 20 specific operations. The Bash primitive running `gh` CLI commands supports everything the GitHub CLI supports, including future additions that require zero changes to Claude Code. As we noted in our [analysis of Anthropic's MCP protocol launch](#), this protocol is designed to be the universal connector for AI agent systems, and Claude Code's implementation shows what mature MCP integration looks like.

The multi-model strategy for tools is worth noting. Claude Code does not use the same model for every operation. The leaked code reveals a tiered approach: a mid-tier **Sonnet** model handles the main reasoning loop, a **Haiku** model handles lightweight operations like conversation title generation, topic classification, security command parsing, and codebase exploration sub-agents, and the flagship **Opus** model of the day (Opus 4.6 in the leaked build) is reserved for ULTRAPLAN's deep planning sessions. This means a single Claude Code session might involve three different models, each selected for the cost/capability tradeoff appropriate to the task.

The tiering outlived the specific model IDs, and that is the part worth copying. Anthropic's own [model configuration docs](#) now put **Claude Opus 5** on the main loop for Max, Team Premium, Enterprise and direct API accounts, **Claude Sonnet 5** for Pro and Team Standard, and a Haiku model on subagents and background work through the

`ANTHROPIC_DEFAULT_HAIKU_MODEL` setting. Every model version named in the leaked constants has since been passed: Sonnet 3.5 retired in October 2025 and Sonnet 4 in June 2026, and Opus 4.6 now sits among Anthropic's [legacy models](#), behind Opus 4.7, Opus 4.8 and Claude Opus 5. Read the leak for the shape of the tiering, not for the version numbers.

Real-world cost data emerged from community testing: a basic repository analysis ("describe what's in this project") required approximately **40 seconds** and cost **\$0.11** in API usage. Complex multi-step tasks with many tool calls can easily reach several dollars. The cost tracker in `src/cost-tracker.ts` monitors input and output tokens separately, accounts for cache read and cache creation tokens, and applies model-specific pricing. Users can check their running cost with the `/cost` slash command at any time during a session.

The **BatchTool** is a particularly clever optimization. Instead of the model making individual tool calls sequentially, the BatchTool allows it to bundle multiple independent operations into a single tool call that executes them in parallel or serial. For example, reading five files can be expressed as a single BatchTool invocation that reads all five simultaneously, rather than five separate Read tool calls that each require a round trip through the TAOR loop. This reduces both latency and token overhead from the tool calling protocol.

There is also a **ToolSearchTool** that implements deferred tool discovery. When Claude Code has access to many MCP servers with dozens of tools each, loading all tool definitions into the system prompt would be expensive and potentially confusing to the model. Instead, tool definitions can be "deferred": only their names are included initially, and the ToolSearchTool lets the model fetch full schemas on demand when it determines a specific tool might be relevant. This keeps the system prompt manageable while maintaining access to a large tool surface.

5. Context Management: Five Compaction Strategies

Context management is arguably the hardest engineering problem in agent systems, and the `src/services/compact/` directory reveals that Anthropic has thrown significant engineering effort at it. Claude Code implements not one, not two, but **five distinct compaction strategies** that work together to prevent context window exhaustion:

1. Snip Compact (`snipCompact.ts`): The simplest strategy. Old messages below a compaction threshold are removed entirely. Fast, lossy, used for messages far enough in the conversation history that their details are unlikely to matter.

2. Microcompact (`microCompact.ts`): Compresses within-turn tool results. When a tool returns a large output (say, the contents of a 500-line file), microcompact can replace the raw content with a reference or summary while preserving the tool call structure. A cached variant (`CACHED_MICROCOMPACT` feature flag) avoids recomputing identical compressions.

3. Auto-Compact (`autoCompact.ts`): The primary compaction strategy. When the conversation approaches a configurable percentage of the context window, auto-compact triggers a full summarization pass. The model itself generates the summary, deciding what information is essential to preserve and what can be compressed. This runs in a **forked subprocess** (separate from the main thread) to avoid blocking the user interaction.

4. Reactive Compact (`reactiveCompact.ts` , gated behind the `REACTIVE_COMPACT` feature flag): A fallback for when the API returns a "prompt too long" error mid-turn. Instead of failing, the system triggers an emergency compaction and retries. This is the safety net that prevents context overflow from crashing the session.

5. Context Collapse (`src/services/contextCollapse/` , gated behind `CONTEXT_COLLAPSE` feature flag): A staged collapse of old messages that is

more aggressive than snip but more selective than full summarization. Think of it as intelligent pruning.

The source code comment in `query.ts` explains how these compose:

```
"Enforce per-message budget on aggregate tool result size. Runs BEFORE microcompact - cached MC operates purely by tool_use_id (never inspects content), so content replacement is invisible to it and the two compose cleanly."
```

This layered approach is not over-engineering. It reflects real production experience with context management. A single strategy always fails at some edge case: auto-compact is too slow for emergency situations, snip is too lossy for recent context, microcompact does not help with conversational bloat. Having five strategies that compose means the system can handle any context pressure pattern.

The practical result: Claude Code can maintain coherent sessions across hundreds of tool calls and thousands of messages without the user ever seeing a context window error. For builders of agent systems, this is the most directly transferable engineering from the leak. Our [deep dive on self-improving AI agents](#) identified memory and context management as the primary bottleneck in recursive agent architectures, and Claude Code's five-strategy approach validates that finding.

There is an additional optimization worth understanding: **content replacement for large tool results**. When a tool returns a very large result (like the contents of a massive file), the system can replace the raw content with a reference pointer, storing the actual content in a separate `ContentReplacementState`. This is tracked per-message with budgets enforced on aggregate tool result size. The source comment explains:

"Runs BEFORE microcompact - cached MC operates purely by tool_use_id

(never inspects content), so content replacement is invisible to it and the two compose cleanly." This means the content replacement layer and the microcompact layer are designed to compose without interfering with each other, a sign of careful architectural layering.

The `sessionMemoryCompact.ts` file handles a related but distinct concern: compacting the session-level memory (MEMORY.md contents and auto-learned patterns) rather than the conversation transcript. This ensures that even persistent knowledge can be managed when it grows beyond useful limits.

The system also supports a manual compaction command (`/compact <focus>`) that lets the user trigger compaction on demand with an optional focus topic, telling the summarizer what information to prioritize preserving. This is an underrated UX feature: when the developer knows they are about to switch from exploration to implementation, they can compact the exploration context with explicit instructions about what to keep, giving them a clean context for the implementation phase.

6. The Six-Layer Memory System

Context compaction handles the within-session problem. The memory system handles the across-session problem. Claude Code implements a six-layer hierarchy that loads contextually at session start, managed through `src/memdir/memdir.ts` and the prompt assembly in `src/constants/prompts.ts` :

Layer 1: Managed Policies. Organization-level rules loaded from remote configuration. These are top-level constraints that the model must always respect, like content policies or behavioral boundaries.

Layer 2: Project Configuration. The contents of `CLAUDE.md` files found in the repository. Claude Code walks up the directory tree, loading these files in order, so a monorepo can have different rules for different packages. These files describe architecture, coding conventions, build commands, and project-specific rules.

Layer 3: User Preferences. Individual settings, communication style preferences, and personalized configurations stored in `~/.claude/`.

Layer 4: Session History. The conversation transcript from the current session, including tool calls and results.

Layer 5: Auto-Learned Patterns. A persistent `MEMORY.md` file that accumulates learned patterns across sessions. The system enforces a strict taxonomy of four memory types (user, feedback, project, reference) with a hard cap of **200 lines** or **25KB**. When the model discovers something useful, it writes to memory with YAML frontmatter specifying the type, and a **Sonnet-powered relevance selector** retrieves up to 5 matching memories per turn.

Layer 6: Real-Time Transcript. The live stream of the current interaction.

The hierarchy establishes a clear priority order: organizational policies trump everything, project configs trump user preferences, and so on. When context needs to be dropped, the system drops from the bottom up: real-time transcript first, then session history gets compressed, then auto-learned patterns get filtered. Organizational policies and project specs are never dropped.

This hierarchical approach solves a problem that flat context stores cannot: **priority-aware information management**. When the context window fills up, a flat store gives you no principled way to decide what to

keep and what to discard. A hierarchy makes the decision mechanical: lower layers get compressed before higher ones.

The system prompt assembly in `src/constants/prompts.ts` (approximately **1,000 lines**) reveals how all six layers get combined into the actual prompt sent to the API. The structure is modular: roughly **60 system prompt components** and **40 system reminders** are conditionally assembled based on the current mode, available tools, connected MCP servers, and active feature flags. The prompt includes embedded data sections with API references for multiple languages (Python, TypeScript, Java, Go, C#, PHP, Ruby, cURL), the complete Claude model catalog with pricing, and mode-specific instructions (learning mode, minimal mode, auto mode).

A key directive from the system prompt is worth noting: **"You should be concise, direct, and to the point"** with target responses of **"fewer than 4 lines of text (not including tool use)."** This is not just a style preference. Short responses reduce output token costs and keep the context window lean. The system prompt is engineering the model's behavior toward efficiency.

That tuning is more delicate than it looks, and Anthropic has since published what happens when it goes wrong. An [April 23, 2026 engineering postmortem](#), written after a month of reports that "Claude's responses have worsened for some users," traced one regression to a hard length limit added to the Claude Code system prompt on April 16, 2026: *"Length limits: keep text between tool calls to ≤ 25 words. Keep final responses to ≤ 100 words unless the task requires more detail."* One internal evaluation showed a **3% drop for both Opus 4.6 and Opus 4.7** from that change alone, and the instruction was reverted on April 20, 2026. Two sentences of prompt text, aimed at precisely the token efficiency described above, measurably degraded the model. It is the clearest public evidence that the system prompt is not packaging around the model but a live part of its

performance surface, which is exactly why the assembly logic in `prompts.ts` is 1,000 lines long.

The `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` constant splits the prompt into two zones: everything before the boundary is globally cacheable across all sessions (tool definitions, safety rules, behavioral instructions), and everything after is per-session (memory contents, project-specific context, MCP server descriptions). This optimization means most of the system prompt is served from Anthropic's prompt cache, saving both latency and cost on every API call. For a tool that makes dozens of API calls per session, the cumulative savings from prompt caching are substantial.

7. Security: 2,592 Lines of Bash Paranoia

The `src/tools/BashTool/bashSecurity.ts` file is **2,592 lines** of security validation, and reading it reveals just how seriously Anthropic takes the problem of letting an AI execute shell commands on a developer's machine.

The security model is multi-layered. The first layer detects **command substitution patterns**, which are shell constructs that could allow command injection:

```
const COMMAND_SUBSTITUTION_PATTERNS = [
  { pattern: /<\(/, message: 'process substitution <() ' },
  { pattern: />\(/, message: 'process substitution >() ' },
  { pattern: /=\(/, message: 'Zsh process substitution =() ' },
  { pattern: /(?:^| [\s;&|])= [a-zA-Z_]/, message: 'Zsh equals expansion ( ' },
  { pattern: /\$\(/, message: '$() command substitution ' },
  { pattern: /\$\{/, message: '${} parameter substitution ' },
  { pattern: /\$\ [/, message: '$ [] legacy arithmetic expansion ' },
```

```
// ... and more  
]
```

Note the Zsh-specific patterns. The comment explains: **"Zsh EQUALS expansion: =cmd at word start expands to \$(which cmd) . =curl evil.com expands to /usr/bin/curl evil.com , bypassing Bash(curl:*) deny rules since the parser sees =curl as the base command, not curl ."** This level of shell arcana awareness reflects real attack research.

The second layer blocks Zsh-specific dangerous commands:

```
const ZSH_DANGEROUS_COMMANDS = new Set( [  
  'zmodload', // Gateway to dangerous module-based attacks  
  'emulate',  // With -c flag is an eval-equivalent  
  'sysopen',  // Opens files with fine-grained control  
  'sysread',  // Reads from file descriptors  
  'syswrite', // Writes to file descriptors  
  // ...  
)
```

The comment on `zmodload` is revealing: **"zmodload is the gateway to many dangerous module-based attacks: zsh/mapfile (invisible file I/O via array assignment), zsh/system (sysopen/syswrite two-step file access), zsh/zpty (pseudo-terminal command execution), zsh/net/tcp (network exfiltration via ztcp), zsh/files (builtin rm/mv/ln/chmod that bypass binary checks)."**

Beyond pattern matching, Bash commands go through **30+ regex validation patterns**, a **tree-sitter AST analysis** for structural command parsing, **here doc extraction** to inspect commands embedded in here-documents, and a **secondary LLM call to a lightweight Haiku model** that extracts and

validates command prefixes. Each command gets a risk classification of LOW, MEDIUM, or HIGH.

This is defense in depth applied to shell security, and it reflects real-world attack attempts. In October 2025, researcher XPN InfoSec discovered **CVE-2025-64755**: the sed command validation had a weak regex that failed to block `echo 'runme' | sed 'w /Users/xpn/.zshenv'`, which could write to shell configuration files. The fix landed in v2.0.31 within a week - [XPN InfoSec](#).

The practical lesson: if you are building an agent system that executes shell commands, your security model needs to account for shell-specific attack vectors that go far beyond simple command injection. Backtick escaping, process substitution, Zsh module loading, heredoc abuse, and sed write commands are all real attack surfaces.

There is also a comment at line 688 of `bashSecurity.ts` that reveals the intentional limitations: **"This simple quote tracker has NO backslash handling."** The developers know where their parser is incomplete and have made deliberate tradeoffs between completeness and performance. A perfect shell parser would essentially need to reimplement bash itself. Instead, the security system uses multiple imperfect layers (regex patterns, tree-sitter analysis, LLM validation) that compensate for each other's blind spots.

The **heredoc extraction** system (`extractHeredocs()` utility) deserves mention because it handles a particularly sneaky attack vector. Commands embedded inside here-documents can bypass simple regex-based security checks because the command does not appear on the same line as the dangerous pattern. The extractor identifies heredoc boundaries, pulls out the embedded commands, and runs them through the same security pipeline as direct commands. This level of thoroughness reflects real-world

adversarial testing, likely from Anthropic's security team actively trying to bypass their own protections.

One more security detail worth highlighting: the system implements a separate check for **git safety**. The source comment in `main.tsx` explains: **"Git commands can execute arbitrary code via hooks and config (e.g., `core.fsmonitor`, `diff.external`), so we must only run them after trust is established."** This means Claude Code defers all git operations until the trust dialog has been accepted, preventing a malicious repository from executing code through git hooks during the initial exploration phase. This kind of defense-in-depth thinking, where even trusted tools like git are treated as potential attack vectors, is the hallmark of production security engineering.

8. The YOLO Classifier and Auto-Mode Permissions

The permission system spans the `src/utils/permissions/` directory with **20+ files** covering every aspect of access control. The system implements multiple permission modes: `plan` (read-only), `default` (ask for confirmation), `acceptEdits` (auto-approve file changes), `auto` (ML-based classification), and `bypassPermissions` (unrestricted).

The most interesting component is the **YOLO Classifier** (yes, that is its actual name in `yoloClassifier.ts`). It powers the "auto" permission mode by making real-time approval decisions using a side-query to Claude. The classifier evaluates each tool invocation and returns a structured decision:

```
type ClassifierDecision = {  
  decision: 'allow' | 'soft_deny' | 'hard_deny'  
  confidence: number
```

```
    reasoning: string
  }
```

A curated set of tools skip the classifier entirely because they are inherently safe. From `classifierDecision.ts` :

```
const SAFE_YOLO_ALLOWLISTED_TOOLS = new Set( [
  FILE_READ_TOOL_NAME,      // Read-only file operations
  GREP_TOOL_NAME,           // Search / read-only
  GLOB_TOOL_NAME,           // Pattern matching
  TODO_WRITE_TOOL_NAME,     // Task management
  TOOL_SEARCH_TOOL_NAME,    // Tool discovery
  // ...
])
```

Write and edit tools get a fast path: allowed within the current working directory, classified outside it. Bash commands always go through the full classifier because of their unbounded risk surface.

An analysis by [UpGuard](#) of **18,470 public** `.claude/settings.local.json` files revealed alarming permission patterns among Claude Code users. **29% allowed the `find` command** (dangerous due to `-exec`), **22% permitted unrestricted `rm`**, **21% allowed unchecked `curl`**, and only **1.1% included any deny rules at all**. These numbers suggest that the YOLO classifier is the last line of defense for most users, since few bother to configure restrictive permission rules.

The permission system also tracks **denial patterns** through `denialTracking.ts`. If the user repeatedly denies a specific tool or action, the system learns to stop asking. And permissions are enforced not just at the tool level but at the **file path level**, with glob pattern matching for

allow/deny rules and path traversal prevention against Unicode and encoding attacks.

The UpGuard findings reveal a fundamental tension in AI tool security. Making a tool powerful enough to be useful (executing arbitrary shell commands, writing to any file) inherently makes it powerful enough to be dangerous. The permission system is Anthropic's attempt to let users choose their own risk tolerance along a spectrum, but the data shows most users maximize convenience over safety. This suggests the YOLO classifier, which makes intelligent automated decisions, is more important than the manual permission configuration, because most users will never configure manual rules.

The system also implements a **bypassPermissions killswitch** (`bypassPermissionsKillswitch.ts`) that can remotely disable the most permissive mode if Anthropic detects a security vulnerability that makes it too dangerous. And there is a **permission explainer** (`permissionExplainer.ts`) that generates human-readable descriptions of what each tool invocation will do, helping users make informed approval decisions rather than blindly clicking "allow."

The IDE bridge adds another dimension to the permission model. When Claude Code runs inside VS Code or JetBrains (via the `src/bridge/` directory), permission prompts are routed to the IDE's native dialog system (`bridgePermissionCallbacks.ts`) rather than the terminal. This provides a better UX (graphical dialogs instead of text prompts) and keeps permission decisions visible in the IDE rather than buried in a terminal window. The bridge uses **JWT authentication** for secure communication between the IDE extension and the CLI process.

9. Sub-Agents: Isolated Forked Intelligence

The `src/tools/AgentTool/` directory reveals how Claude Code implements sub-agents. When the model decides a task benefits from delegation (exploring a large codebase, verifying a complex change, planning an implementation), it can spawn a sub-agent with its own isolated context.

Agent definitions can be loaded from `~/.claude/agents.json` or specified inline:

```
type AgentDefinition = {  
  agentType: string  
  description: string  
  instructions?: string  
  mcpServers?: string []  
  model?: string  
  tools?: string []  
  toolFilters?: { allow: string [], deny: string [] }  
  maxTurns?: number  
  maxBudgetUsd?: number  
}
```

The key property is **isolation**. Each sub-agent gets:

- An **independent context window** (not sharing with the parent)
- A **separate token budget** (preventing runaway cost)
- **Restricted tool access** (cannot spawn their own sub-agents in most modes)
- **Independent compaction triggers** (manages its own context)
- Its own **MCP server connections** (created fresh, cleaned up on exit)

The parent receives only a summary of the sub-agent's work, protecting the main context from pollution. This is critical. Without isolation, a sub-agent that reads 50 files would flood the parent's context with file contents it does not need.

Built-in agent types include **ExploreAgent** (read-only codebase exploration using a lightweight model) and a **VerificationAgent** that validates task completion. The leaked code also shows a **COORDINATOR_MODE** feature flag for multi-agent coordination, where a master coordinator directs parallel workers through shared scratch directories.

Behind another feature flag (`AGENT_MEMORY_SNAPSHOT`), there is infrastructure for persistent agent memory. Custom agents can maintain their own `MEMORY.md` files at `~/.claude/agent-memory/<name>/MEMORY.md` , accumulating knowledge across sessions. This transforms one-shot sub-agents into persistent workers that learn from every interaction.

The **task verification pattern** is an interesting safeguard built into the agent system. A comment in `TaskUpdateTool.ts` reveals: **"You just closed out 3+ tasks and none of them was a verification step. Before writing your final summary, spawn the verification agent..."** This means the system automatically triggers a verification sub-agent when the main agent completes multiple tasks without verifying any of them. The verification agent reviews the work independently, in its own isolated context, before the results are presented to the user. This is a form of self-checking that catches errors the main agent might miss due to confirmation bias.

The sub-agent system also supports **model override per agent**. The `ExploreAgent` uses a lighter model for codebase exploration (where speed matters more than depth), while the main agent uses the full-capability model. This multi-model approach within a single session is a meaningful cost optimization: exploration queries that read files and search patterns do not need Opus-level reasoning, and using Haiku or Sonnet for these

operations can reduce the cost of exploration-heavy sessions by 50% or more.

For teams building multi-agent systems, the isolation pattern is the most transferable insight. We covered the broader landscape of multi-agent architectures in our [guide to CrewAI and multi-agent orchestration](#), and Claude Code's approach of forked processes with hard context boundaries is the most production-proven pattern. The key lesson is that sub-agents should be cheap to create, fully isolated, and return only summaries. Any architecture that shares context between parent and child agents will eventually hit context window limits during complex tasks.

10. Hidden Features: KAIROS, ULTRAPLAN, and the Buddy System

The leaked code contains **32 compile-time feature flags** that Bun's dead code elimination strips from public builds. These flags are scattered throughout `main.tsx` (which contains over **60 `feature()` calls**) and reveal Anthropic's product roadmap for Claude Code.

KAIROS: The Always-On Assistant

KAIROS transforms Claude Code from a reactive tool into a **proactive, always-on assistant** that continuously monitors the development environment. The code reveals a system that receives `<tick>` prompts at regular intervals, each with a **15-second blocking budget**, and decides whether to act on what it observes.

KAIROS maintains **append-only daily logs** at

`~/.claude/.../logs/YYYY/MM/DD.md`, creating a running record of development activity. It includes exclusive tools not available in standard

Claude Code: `PushNotification` (alert the user to something important), `SendUserFile` (proactively deliver files), and `SubscribePRTool` (automatically monitor GitHub pull request activity and react to changes).

The architecture supports **cross-session continuity** with cloud-based midnight boundary handling, meaning KAIROS can maintain state across days. This is not a simple background watcher. It is a persistent development partner that can notice a failing CI pipeline and start investigating before you see the notification, watch a pull request review and prepare responses to comments, or monitor dependency updates and flag breaking changes.

We analyzed this pattern of ambient AI assistance in our [guide to vibe automation](#), where the core insight is that the most productive AI systems operate continuously in the background rather than waiting for explicit commands. KAIROS is the clearest production implementation of this pattern.

ULTRAPLAN: Remote Deep Thinking

ULTRAPLAN offloads complex planning to **Cloud Container Runtime (CCR) sessions** running the flagship Opus model (**Opus 4.6** in the leaked build, a slot now held by **Claude Opus 5**) for up to **30 minutes** per session. The code references a special sentinel value `__ULTRAPLAN_TELEPORT_LOCAL__` for local testing and a configuration key `tengu_ultraplan_model` for model selection.

This solves a fundamental constraint. Some problems (large-scale refactoring, architecture migration, complex debugging) require more reasoning time than a real-time interactive session can provide.

ULTRAPLAN lets the model spend 30 minutes thinking deeply, then presents results through a **browser-based approval interface** for human

review. The remote execution also means planning computation does not block the local machine.

autoDream: Background Memory Consolidation

The `src/tasks/DreamTask/DreamTask.ts` file reveals a background memory consolidation system inspired by UC Berkeley's "Sleep-time Compute" research. The DreamTask comment explains: **"Background task entry for auto-dream (memory consolidation subagent). Makes the otherwise-invisible forked agent visible in the footer pill."**

The system runs as a forked sub-agent through **four phases**: Orient, Gather Signal, Consolidate, Prune. Three gates must be satisfied before it triggers: a **24-hour time gate**, a **5-session minimum** since the last dream, and **consolidation lock acquisition** (preventing concurrent dreams). Output is capped at 25KB. The server-side feature flag is `tengu_onyx_plover`, and the research it is based on (arXiv:2504.13171) showed a **5x reduction in test-time compute** at equal accuracy.

The Buddy System: Tamagotchi for Developers

The most surprising finding is the `src/buddy/` directory, implementing a full **Tamagotchi-style companion pet**. This is not a joke or Easter egg. It is a carefully engineered engagement feature.

The companion system uses a **Mulberry32 PRNG** (pseudo-random number generator) seeded from a hash of the user's ID, making it fully deterministic:

```
// Mulberry32 - tiny seeded PRNG, good enough for picking ducks
function mulberry32(seed: number): () => number {
```

```
let a = seed >>> 0
return function () {
  a |= 0
  a = (a + 0x6d2b79f5) | 0
  let t = Math.imul(a ^ (a >>> 15), 1 | a)
  t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t
  return ((t ^ (t >>> 14)) >>> 0) / 4294967296
}
```

The comment "good enough for picking ducks" is delightful. The system supports **18 species** (duck, goose, blob, cat, dragon, octopus, owl, penguin, turtle, snail, ghost, axolotl, capybara, cactus, robot, rabbit, mushroom, chonk) across **5 rarity tiers** with specific weights:

```
export const RARITY_WEIGHTS = {
  common: 60,
  uncommon: 25,
  rare: 10,
  epic: 4,
  legendary: 1,
}
```

Each companion has procedurally generated stats (DEBUGGING, PATIENCE, CHAOS, WISDOM, SNARK), a hat (none, crown, tophat, propeller, halo, wizard, beanie, tinyduck), eye style (using Unicode characters like ·, ✦, ✕, ◎, @, °), and ASCII art sprites with idle fidget animations.

The species names are encoded as character codes to avoid triggering internal codename canary checks:

```
// One species name collides with a model-codename canary in excluded-strings.txt
export const capybara = String.fromCharCode(0x63,0x61,0x70,0x79,0x62,0x61,
```



This reveals that "capybara" is also an internal Anthropic model codename, and the build system has automated checks that scan for leaked codenames. The buddy system works around this by constructing species names at runtime rather than including the string literals.

Only the **soul** (name and personality, generated by Claude on first hatch) persists in config. The bones (rarity, species, stats) are regenerated from the user ID hash every time, so **"users can't edit their way to a legendary."** This is anti-cheat engineering applied to a virtual pet in a coding tool.

Other Discoveries

Undercover Mode: Protecting Internal Secrets

The `utils/undercover.ts` file implements a system that injects special instructions into the system prompt when `USER_TYPE='ant'` (Anthropic employee). These instructions require that commit messages and PR bodies must not contain internal information: model codenames (which use animal names like "Capybara," "Tengu," "Fennec"), unreleased model version numbers, internal repository names, and internal tooling references.

The capybara naming conflict shows up concretely in the buddy system.

The species name is encoded as character codes

(`String.fromCharCode(0x63,0x61,0x70,0x79,0x62,0x61,0x72,0x61)`) rather than the literal string "capybara" because the build system has automated checks in `excluded-strings.txt` that scan for leaked codenames. A comment in `buddy/types.ts` explains: **"One species name collides with a**

model-codename canary in excluded-strings.txt. The check greps build output (not source), so runtime-constructing the value keeps the literal out of the bundle while the check stays armed for the actual codename."

This is operational security applied to build artifacts. It reveals that Anthropic maintains a list of strings that must never appear in public builds, and the build system enforces this automatically. The buddy system team had to work around their own security system just to name a virtual pet species.

Anti-Distillation Defense

The system injects **decoy tool definitions** into API requests to poison training data from competitors who might scrape Claude Code's outputs to train their own models. This is an offensive countermeasure against model distillation. The Hacker News community flagged this as a notable competitive defense mechanism, and it raises interesting questions about the arms race between AI labs. If competitors are training models on the outputs of each other's products (which is widely suspected), injecting plausible-looking but subtly wrong tool definitions could introduce systematic errors in the resulting models.

Sentiment Detection and Telemetry

Hardcoded regex patterns detect user frustration indicators (specific profanity and frustration keywords). When triggered, the system logs the event through Datadog telemetry. HN commenters found this ironic for a company that builds language models (surely Claude could detect frustration better than regex), and noted implementation gaps (the patterns do not catch variations and combinations). The system also maintains **120+ environment variables** (including `DISABLE_COMMAND_INJECTION_CHECK` marked

"DANGEROUS" and `CLAUDE_CODE_PERFETTO_TRACE` for performance tracing) and exposes **18 unreleased API beta headers** for features like extended thinking, 1M token context, structured outputs, and effort-level control.

The Hooks System

One of the more powerful extensibility mechanisms is the **hooks system**, which allows deterministic shell scripts to execute at lifecycle events. The code supports hooks at eight lifecycle points: `SessionStart` , `UserPromptSubmit` , `PreToolUse` , `PermissionRequest` , `PostToolUse` , `PostToolUseFailure` , `PreCompact` , and `SessionEnd` . The critical detail: if a hook exits with **code 2**, it blocks progression. This means organizations can implement mandatory compliance checks, code signing requirements, or audit logging that cannot be bypassed by the model.

Hooks are configured through settings files, not by the model itself, which is an important security boundary. The model cannot install its own hooks. Only the user or organization administrator can. This prevents a compromised or misbehaving model from creating persistence mechanisms that survive session boundaries.

11. Code Quality: The "Vibe Coded" Debate

The Hacker News thread turned into a heated debate about code quality, and the discussion deserves honest treatment because it reveals real tensions in AI-assisted software development - [Hacker News](#).

The most cited example is `src/cli/print.ts` . Our local analysis confirms it is **5,594 lines** in a single file. Hacker News commenters reported the primary function within it has **12 nesting levels** and approximately **486 branch points** of cyclomatic complexity. Multiple people characterized the

entire codebase as **"vibe coded"**: generated or heavily assisted by AI without traditional software engineering discipline.

Another data point: `main.tsx` is approximately **50KB** (the commenters reported a bundled version at 785KB). The file contains the entire CLI setup, command registration, feature flag gating, permission initialization, session management, and startup optimization in a single file.

But first-principles analysis demands we ask: **what is code quality actually measuring?**

Code quality standards exist to make software maintainable by humans. A 5,594-line file is painful for a human to navigate because human working memory is limited. Deep nesting makes code hard to reason about because humans lose track of context after 3-4 levels. But these constraints are weaker when the primary author and maintainer is an AI that can regenerate understanding of any code it has previously written.

The counterargument is equally valid: AI-generated code still needs to be debugged by humans when things go wrong. It still needs to be reviewed by humans for security vulnerabilities. And when the AI generates a subtle bug in line 4,200 of a 5,594-line file, a human still has to find it.

The real lesson is nuanced. Claude Code's architecture shows excellent engineering judgment in the areas that matter most: the TAOR loop is clean and minimal, the permission system is thorough, the memory hierarchy is well-designed, the tool abstractions are composable, and the startup optimization is sophisticated. The areas where quality drops (the massive print function, the oversized main file) are exactly the areas where AI-assisted development struggles: UI rendering and orchestration logic, where the requirements are complex but not algorithmically deep, and where code grows organically through incremental additions.

Anthropic reportedly releases **60-100 internal builds daily** with engineers averaging **5 PRs per day**. At that velocity, traditional code quality practices like careful refactoring, modular decomposition, and comprehensive code review are in tension with shipping speed. The codebase reflects this tension honestly.

There is a deeper structural point here about how AI-written codebases evolve differently from human-written ones. When a human writes a function and it grows beyond 200 lines, the cognitive load becomes painful and they refactor. When an AI writes a function and it grows to 5,000 lines, there is no cognitive load, and the economic incentive to refactor is weaker (time spent refactoring is time not spent shipping features). This creates a different technical debt profile: the code works, the tests pass, the product ships, but the files are massive and the nesting is deep.

The Pragmatic Engineer interview revealed that the team's prototyping velocity was extreme: **10-20+ iterations per feature**, with the TodoWrite UI going through approximately **20 variations over two days**. This rapid iteration style favors adding and modifying over restructuring. Each iteration adds a new branch, a new condition, a new edge case, and the result is exactly what we see in `print.ts`: a function that handles every edge case correctly but has accumulated the archaeological layers of 20 iterations without a consolidation pass.

Whether this matters depends entirely on who maintains the code. If Claude Code continues to be **90% self-written**, and if the AI can navigate 5,594-line files without difficulty, then the traditional arguments for modular decomposition are weaker. But if a new human engineer needs to debug a production issue at 3 AM, the 12 nesting levels become a real problem. The pragmatic answer is probably that both AI and human readability matter, and the codebase will eventually need a consolidation pass, but not before the product-market fit is established and the velocity pressure decreases.

For the broader AI-assisted development community, this codebase is a useful data point. It shows what happens when you let an AI write most of a large codebase at high velocity: the architecture can be excellent (the TAOR loop, the permission hierarchy, the compaction strategies) while the implementation can be messy (the massive files, deep nesting). The question going forward is whether new tooling can bridge this gap, perhaps AI-powered refactoring tools that can consolidate 20 iterations into clean modular code after the fact. We explored related patterns of AI-assisted development quality in our [Claude Code pricing and alternatives guide](#).

12. What This Means for Agent Harness Design

Stepping back from the specific details, the Claude Code leak validates several principles about production agent harness design that were previously theoretical.

Intelligence belongs in the model, not the harness. The thin TAOR loop is the clearest expression of this principle. Most agent frameworks (LangChain, CrewAI, AutoGen) invest heavily in orchestration: chains, graphs, routers, planners. Claude Code inverts this. The orchestration layer is intentionally thin because the model handles planning internally. The practical benefit is fewer round trips: the model can batch multiple logical steps into a single response, while framework-heavy systems make separate LLM calls for each step.

Primitives compose better than integrations. Four tools that compose into anything (read, write, execute, connect) are more powerful than 400 single-purpose integrations. The maintenance burden of pre-built integrations scales linearly. Composable primitives have constant maintenance cost. The MCP extension mechanism is the escape hatch for specialized needs.

Context management requires multiple strategies. Five compaction approaches (snip, microcompact, auto-compact, reactive compact, context collapse) exist because no single strategy handles all scenarios. This is the most directly actionable finding for agent builders: do not try to solve context management with one approach.

Safety must be structural, not prompting. The permission system uses prompt instructions, an ML classifier, tool-level access controls, tree-sitter AST analysis, regex validation, a secondary LLM call, and path traversal prevention. Seven layers. Because no single layer is reliable on its own.

Design for shrinkage. The "delete code when models improve" philosophy means the harness is a temporary wrapper around temporary model limitations. Every piece of scaffolding is a bet that the model cannot handle that task yet. As models improve, the scaffolding gets removed. This is counterintuitive for engineers trained to build durable systems, but it is the correct approach for systems at the intersection of software engineering and rapidly advancing model capability.

For teams evaluating whether to build or buy their agent infrastructure, platforms like o-mega.ai offer a managed alternative: rather than building a harness around a single model, they provide an **AI workforce platform** where multiple specialized agents handle different tasks through unified orchestration. This trades control over the harness for speed of deployment and breadth of capability.

The terminal UI is its own engineering achievement. The React/Ink rendering layer uses Meta's Yoga flexbox engine for layout, implements a custom React reconciler with **double-buffered screen rendering, cell-level blitting** (only updating changed cells rather than redrawing the entire screen), interned style and character pools for O(1) comparisons, hardware scroll region optimization, and full text selection with word and line modes. This is not a simple text output. It is a full terminal UI framework that gives

Claude Code the responsiveness of a native application while running in a terminal emulator.

The analytics architecture is carefully separated from the core loop. A decoupled analytics sink with gateway detection (recognizing proxy services like LiteLLM, Helicone, and Portkey) ensures that telemetry does not slow down the main execution path. Event logging uses a type-safe wrapper (`logEvent`) with a deliberately named parameter type:

`AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS` . This long type name is not poor engineering. It is an intentional friction mechanism that forces developers to pause and verify they are not accidentally sending code content or file paths to the analytics system, a meaningful privacy safeguard.

Startup profiling reveals the optimization obsession. The

`startupProfiler.ts` utility runs on **0.5% of external user sessions** (sampled), collecting memory snapshots and timing data for each startup phase. The profile checkpoints fire at module entry, after MDM reads, after keychain prefetch, after GrowthBook initialization, and after tool registration. This data feeds back into the next round of startup optimization, maintaining the fast cold-start that makes Claude Code usable as a frequently invoked CLI tool.

13. Where This All Goes Next

The Claude Code leak is a snapshot of agent architecture at a specific moment in early 2026. The trajectory it reveals matters more than the snapshot.

The feature flags tell the story. **KAIROS** points toward always-on ambient AI assistance. **ULTRAPLAN** points toward offloading complex reasoning to remote compute. **COORDINATOR_MODE** points toward multi-agent teams

with shared state. **autoDream** points toward persistent memory consolidation that mimics biological sleep. The **Buddy System** points toward emotional engagement and retention mechanics in developer tools.

These are not disconnected experiments. They form a coherent roadmap: Claude Code evolving from an on-demand coding assistant into a **persistent development partner** that runs continuously, maintains long-term memory, coordinates with other agents, and builds a relationship with its user through gamified engagement.

The progression is structural, not incremental. Each feature flag represents a qualitative capability jump. Today's Claude Code waits for you to ask it something. Tomorrow's KAIROS-enabled Claude Code watches your development environment and acts proactively. Today's Claude Code runs a single planning step in real time. Tomorrow's ULTRAPLAN-enabled version offloads to a remote container running Opus for half an hour and returns with a comprehensive implementation strategy. Today's Claude Code forgets everything between sessions (except what fits in MEMORY.md). Tomorrow's autoDream-enabled version consolidates its memories while you sleep, organizing knowledge from your last five coding sessions into coherent patterns.

The competitive implications are significant. OpenAI has open-sourced [Codex](#), Google has released Gemini CLI, and dozens of startups are building AI coding assistants. The surface-level feature sets are converging: all of them can read files, write code, run commands, and talk to APIs. But the leaked feature flags reveal where Anthropic is placing its next bets: ambient persistence (KAIROS), deep reasoning offload (ULTRAPLAN), biological-inspired memory (autoDream), multi-agent coordination (COORDINATOR_MODE), and emotional engagement (BUDDY). These are not features that can be replicated by wrapping a model in a simple CLI. They require the kind of deep product engineering that the 512,000 lines of code represent.

As we documented in our [analysis of AI market power consolidation](#), the AI agent market is bifurcating between commodity model providers and differentiated application layers. Claude Code's leaked roadmap confirms that Anthropic sees the application layer (the harness, the product, the experience) as the competitive moat, not just the model underneath it.

The anti-distillation defenses suggest something about competitive dynamics: that as model capabilities converge across labs (which they are, as [we documented in our AI market power consolidation analysis](#)), the agent harness becomes the differentiator. How you manage context, handle permissions, design the tool system, and implement memory: these become the competitive moat, not the model itself.

For the open-source community, this leak accelerates the conversation about agent harness standards. The frameworks (LangChain, CrewAI, AutoGen) now have a concrete production reference to measure against. Independent builders have a blueprint for permission systems, memory hierarchies, and tool architectures validated at scale. And every AI company shipping developer tools has a cautionary tale about build pipeline hygiene.

The code is out there. **1,900 files. 512,000 lines. 32 feature flags.** The architecture is documented in detail. The question now is what the community builds with this knowledge, and how fast the next generation of agent harnesses can close the gap between open-source prototypes and production-grade systems like this one.

The most underappreciated finding from the leak may be the **Skills system** (`src/skills/`). Skills are prompt macros with optional lifecycle scoping and tool restrictions. They can be discovered from five sources, invoked via slash commands or auto-triggered based on context. Combined with the Plugin system (which bundles skills, hooks, and MCP servers with user-configurable variables and version management), Skills represent a secondary extension layer beyond MCP. Where MCP extends Claude Code's

capabilities (what it can do), Skills extend its behavior (how it approaches tasks). The combination creates a deeply customizable system.

The **Coordinator Mode** feature flag hints at the future of multi-agent coding. When enabled, a master coordinator can direct parallel worker agents through XML protocols and shared scratch directories, supporting research, synthesis, implementation, and verification phases. Combined with the **Agent Teams** experimental feature (peer-process agents coordinating through shared task lists with self-claiming mechanisms, bidirectional messaging, and broadcast notification), this paints a picture of Claude Code evolving from a single-agent tool into a multi-agent development team. That direction stopped being speculative on June 2, 2026, when Anthropic shipped [dynamic workflows](#): JavaScript functions that let Claude "write its own harness on the fly," spawning and coordinating subagents, choosing which model each one runs, and optionally giving each its own git worktree. The leaked flag described a coordinator baked into the harness. What shipped goes a step further and lets the model generate the coordinator per task.

There is also a **UDS Inbox** feature (Unix Domain Socket) for cross-session IPC, allowing separate Claude Code instances to message each other. And a **Daemon Mode** for background session management with process listing, log viewing, attach/detach, and kill capabilities. Together with KAIROS, these features suggest a future where Claude Code runs as a persistent service rather than an on-demand CLI invocation.

The fundamental insight from reading all 512,000 lines is simple: **the harness should be as thin as possible, the safety should be as deep as possible, and everything else should live in the model.** That is the design principle. Everything else is implementation detail.

But there is a second insight that is equally important: **the harness is not just code around a model. It is an entire product surface.** The terminal UI,

the permission dialogs, the startup optimization, the buddy system, the memory persistence, the IDE integration, the hooks system, the analytics pipeline, the cost tracker, the compaction strategies, the agent isolation, the MCP extensibility: all of these are product features, not just engineering infrastructure. The leaked code reveals that building an AI coding assistant is roughly 10% model integration and 90% product engineering. The TAOR loop is 50 lines. Everything else is the product.

That ratio, 50 lines of core intelligence wiring and 512,000 lines of product engineering, is the most important number in the entire leak. It tells you where the real work is in building AI agent systems, and it is not where most people think it is.

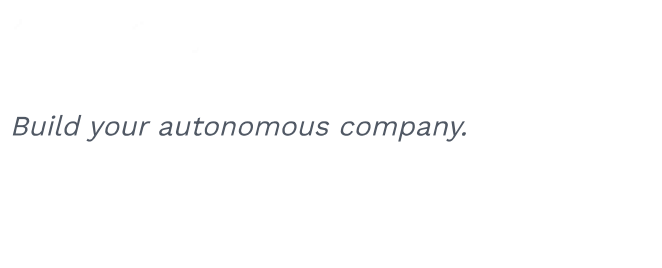
This analysis reflects the Claude Code architecture as captured in the March 2026 source leak. Model names, defaults and lifecycle status were last verified against Anthropic's published documentation in September 2026. AI agent tooling evolves rapidly. Verify current details before making architectural decisions based on this analysis.

Build Your Own AI Agent Workforce

Stop reading about AI agents, deploy your own. O-mega lets you build and manage a full AI workforce today.

Start Building Free

// Turn your vision into an empire <3



Build your autonomous company.

COMPANY

[Plans](#)

[Enterprise](#)

[About](#)

[Articles](#)

LEGAL

[Privacy Policy](#)

[Terms of Service](#)

PRODUCT

[Founden](#)

[Suprsonic](#)

[Suprbrowser](#)

RESOURCES

[Documentation](#)

[AI Agent Index](#)

[Brand Assets](#)

[Support](#)

[Contact](#)



© 2026 O-mega Enterprise, Inc.. All rights reserved.